

I Want My App *That Way*: Reclaiming Sovereignty Over Personal Devices

Konrad Kollnig*
konrad.kollnig@cs.ox.ac.uk
Department of Computer Science,
University of Oxford
Oxford, United Kingdom

Siddhartha Datta*
siddhartha.datta@cs.ox.ac.uk
Department of Computer Science,
University of Oxford
Oxford, United Kingdom

Max Van Kleek
emax@cs.ox.ac.uk
Department of Computer Science,
University of Oxford
Oxford, United Kingdom

ABSTRACT

Dark patterns in mobile apps take advantage of cognitive biases of end-users and can have detrimental effects on people's lives. Despite growing research in identifying remedies for dark patterns and established solutions for desktop browsers, there exists no established methodology to reduce dark patterns in mobile apps. Our work introduces *GreaseDroid*, a community-driven app modification framework enabling non-expert users to disable dark patterns in apps selectively.

CCS CONCEPTS

• **Human-centered computing** → **Systems and tools for interaction design**; *Accessibility systems and tools*; • **Security and privacy** → *Software reverse engineering*.

KEYWORDS

dark patterns, digital distraction, digital self-control, mobile app, program repair

ACM Reference Format:

Konrad Kollnig, Siddhartha Datta, and Max Van Kleek. 2021. I Want My App *That Way*: Reclaiming Sovereignty Over Personal Devices. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3411763.3451632>

1 INTRODUCTION

As our primary interfaces to both the digital world and to one another, mobile phones have become more than indispensable accessories, but extensions to our very beings that allow us operate as citizens within a distributed, global society [1]. Yet, the very mobile apps that now enable people to do so much are also being used as leverage by app developers, platforms, and services to exploit end-users, as evidenced by an increasing variety of user-hostile elements situated within services that provide this essential digital

functionality [2]. Such exploitation ranges from behavioural design (nudging people to take certain actions, such as making systems addictive), to exploiting users' psychological weaknesses, to features that directly harm the user such as by undermining their privacy either by giving up their personal information directly or by tracking them behind the scenes [3–5]. Even if apps do not pose direct harm to the majority of users, they often fail to account for the diversity and sensitivities of the most vulnerable groups of users [6]. Unfortunately, the tools for users to change and challenge their essential technological infrastructure are currently limited. Much of the essential software infrastructure is developed and deployed by powerful international tech companies, with limited means for user participation and negotiation in software design. What if users were given the ability to overcome user-hostile features in technology, thereby putting power back into the hands of its users?

This article explores how one specific example of user-hostile design in mobile apps, namely *dark patterns*, can be alleviated. Dark patterns are carefully-designed interfaces that induce a user to act with specific intent of the developer and widely used in mobile apps. Studies that inspect UI elements on apps indicate high prevalence of dark patterns in web and software applications, typically covert, deceptive or information hiding in nature [7–9]. Dark patterns can have detrimental effects on data protection and consumer protection through the systematic exploitation of users' cognitive biases [10–13]. This is particularly harmful to children, the aged, individuals with disabilities [14], and the broader population [15]. An intentionally-designed feature for one user may be considered a prohibitive bug for another.

This paper contributes to reducing the impact of dark patterns in mobile apps:

- Our *conceptual contribution* is a system for app modification as a way to reduce dark patterns in mobile apps. We believe that a community-driven approach can allow even non-expert users to modify apps in a way that fits their needs.
- Our *technical contribution* is a working prototype of a user-scripting toolkit that manifests our general contribution to minimize dark patterns. We introduce sample patches with a case study of the Twitter app and show that dark patterns can be easily removed.

Whilst this article evaluates dark patterns, *GreaseDroid* is by far not limited to this example of user-hostile design. It can easily be used to modify other aspects of apps, including addictive design patterns and concerning privacy practices.

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8095-9/21/05...\$15.00

<https://doi.org/10.1145/3411763.3451632>

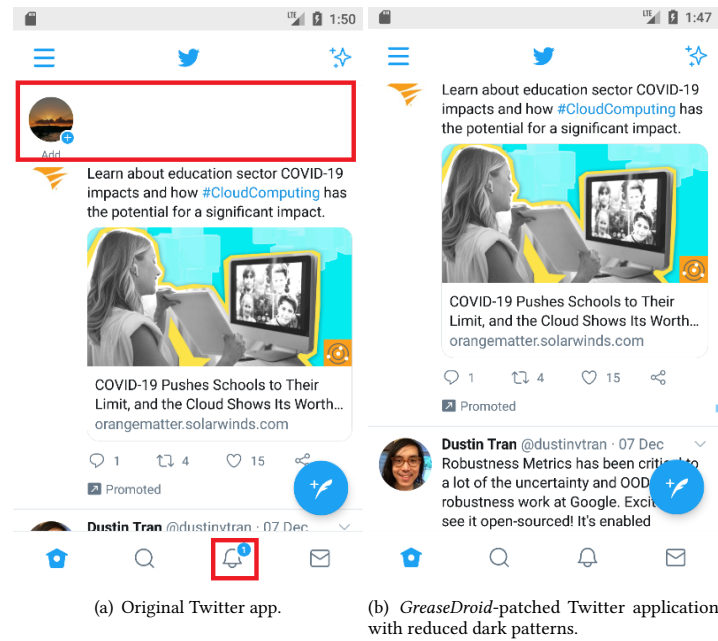


Figure 1: GreaseDroid enables the removal of dark patterns (highlighted in red) in Android apps. Compared to the default Twitter app (left), stories and notifications have been disabled to reduce distractions in the patched version of the app (right).

2 RELATED WORK

As researchers in HCI, we strive to understand users and craft interfaces and systems for them. Unfortunately, user-focused research can be exploited against our users, with the design of interfaces that abuse cognitive and psychological vulnerabilities. It is proven that interfaces can inductively change a user’s cognitive functioning [16–18]. Maliciously-crafted interfaces can thus induce precise cognitive functions intended by the developer, manifesting as digital addiction, digital persuasion, digital nudging, gamification, data exploitation, and dark patterns [19].

This work focuses on *dark patterns*, which can be defined as carefully-designed interfaces that induce a user to act with specific intent of the developer based on a user’s cognitive biases. Gray *et al.* provide a 5-class taxonomy of dark patterns [20]. (1) *Interface Interference* elements manipulate the user interface such that certain actions are induced upon the user compared to other possible actions. (2) *Nagging* elements interrupt the user’s current task with out-of-focus tasks, usually in the form of a choice-based popup to redirect a user towards another goal. (3) *Forced Action* elements introduce sub-tasks forcefully before permitting a user to complete their desired task. (4) *Obstruction* elements introduce subtasks with the intention of dissuading a user from performing an operation in the desired mode. (5) *Sneaking* elements conceal or delay information relevant to the user in performing a task.

Given the prevalence of such interface patterns, more and more users wish to exercise *digital self-control* to protect themselves. Despite the increasing body of literature on digital self-control, along with a wide array of anti-distraction apps, many users still struggle to exert meaningful control over their digital device use [3]. It has

been shown that interventions against harmful design patterns, such as hiding the Facebook feed in the desktop browser, can be effective at reducing usage time, and making users feel more in control of their device use [4]. Yet, the limited ways for intervention on mobile devices are a major limitation of current digital self-control research and apps.

There is a long history of tools that try to fix deficiencies in the design of programs on *desktops*, notably on websites. One of these tools is the browser extension Greasemonkey. This tool lets users choose from a wide array of userscripts to change the design or functionality of a given website to fit their needs. For instance, users can install userscripts to remove the feed from Facebook or increase the readability of Wikipedia. Similar technologies as used in Greasemonkey are the backbone of ad blockers, which are widely in use [21].

Greasemonkey and its variants can be considered examples of program repair tools. Program repair methods [22–24] are concerned with the improvement of software and removal of bugs or errors after deployment. Dark patterns can pose functional hindrances, and could be considered as “bugs”. Dark patterns may leverage cognitive biases of the individuals that exceed the extent intended by or even beyond the intention of the developer, hence requiring certain dark patterns to be selectively mitigated.

There exist some solutions for mobile devices to enhance the properties of apps. Methodologically, these solutions either 1) modify the operating system (e.g. Cydia Substrate [25], Xposed Framework [26], ProtectMyPrivacy [27], or TaintDroid [28]), 2) modify apps (e.g. Lucky Patcher [29], apk-mitm [30], Joel *et al.* [31], Droid-Force [32], RetroSkeleton [33], AppGuard [34], I-ARM-Droid [35],

Aurasium [36]), or 3) use System APIs (e.g. VPN-based ad blockers, such as AdGuard [37] and TrackerControl [38]).

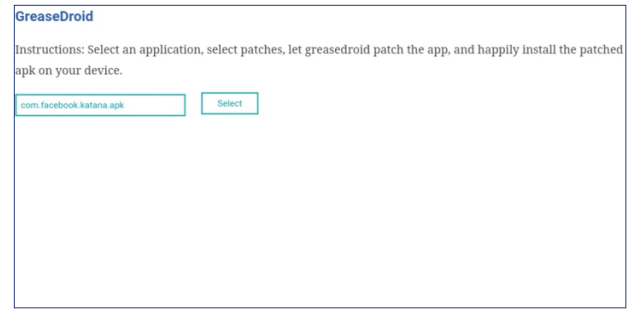
All of these solutions come with certain limitations. Whilst modifying the operating system can in principle make arbitrary modifications to the behaviour of apps, these usually rely on device vulnerabilities and are a moving target for device manufacturers. Operating system modifications can pose security risks, potentially void warranty, and are usually infeasible for non-expert users. By contrast, the use of System APIs might often be the most straightforward approach for a non-expert user, operating in the familiar space of the user's own smartphone. This also poses a major limitation because only what is permitted by the smartphone operating system can be realised. For instance, the removal of the newsfeed from the Facebook app has not been accomplished through System APIs.

In app modification on Android, some transformation is applied to an app used by the user, e.g. the removal of the newsfeed from the Facebook app. App modification offers ease of use (installing custom apps on Android is supported by the operating system), and allows for arbitrary modifications of a provided app (only limited by the constraints of the operating system). As such, it combines benefits of system modification and System APIs. Despite this, there exist hardly any solutions used in practice using app modification. One exception is the app cracking tool Lucky Patcher that allows to remove ads and paywalls from apps. Another exception is *apk-mitm* to remove certificate pinning from apps and allow to intercept encrypted network traffic. Some developers have published modified versions of popular apps, including Facebook [39] and YouTube [40] (removing ads and other distracting functions). Unfortunately, such modified apps rely on the continued support of the developers, may break over time (e.g. in case of server-side updates), and exist only for a few select apps.

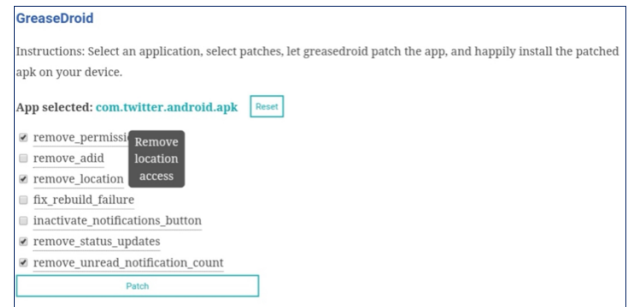
3 GREASEDROID

To date, there exist no tool for non-expert smartphone users to remove dark patterns from their mobile apps. We propose a patching paradigm based upon app modification, called *GreaseDroid*, that enables user-scripting to reduce dark patterns in mobile apps. *GreaseDroid* acts as a tool that contains the assets needed for users to act autonomously in improving apps for their own wellbeing. We retain a high level of abstraction for flexible implementation and additional modules; our implementation in code is covered in section 5. Users go through three main phases (see in Figure 3a). A user (1) selects an app, (2) applies a set of patches, and then (3) re-installs the apps on their phone. We share our code on GitHub¹. **(1) App selection.** In the first step, the user selects the app they want to patch. *GreaseDroid* shall allow the user to select arbitrary apps in the form of apk files (the standard format for apps on Android). For increased ease of use, *GreaseDroid* could directly fetch the latest version of the app from Google Play, or another third-party app store. This would ensure that the user only ever patches the latest version of an app, and minimise unnecessary hassle of finding the apk file themselves. Due to potential violations of Google's Terms & Conditions, we have not yet implemented such an approach.

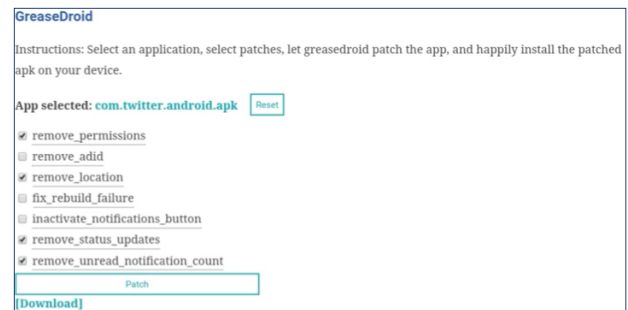
¹See <https://github.com/OxfordHCC/greasedroid>.



(a) Step 1: User selects an app.



(b) Step 2: User selects patches.



(c) Step 3: User installs patched app.

Figure 2: Our implementation enables non-expert users to remove dark patterns from apps in three simple steps.

(2) Patching. In the second phase, the user selects a set of patches to apply to the chosen app. These patches are developed by expert community users, that we call *patch developers*. Once the user has chosen a set of patches, these are applied to the selected app and modify the app code (e.g. `smali` assembly code compiled libraries) or resources (e.g. `xml` layout files, images, sounds).

Patches can be created in at least two ways, either as patch scripts or byte masks. Patch developers can create patch scripts by first decoding the app with `apktool`. They then sift through assembly code (stored as `smali` files and compiled libraries) and other resources (e.g. `xml` layout files, images, sounds) to identify potential code modifications for dark patterns. If the patches are generalizable over a number of apps, these are considered *app-agnostic*, else *app-specific*. Our *GreaseDroid* implementation uses

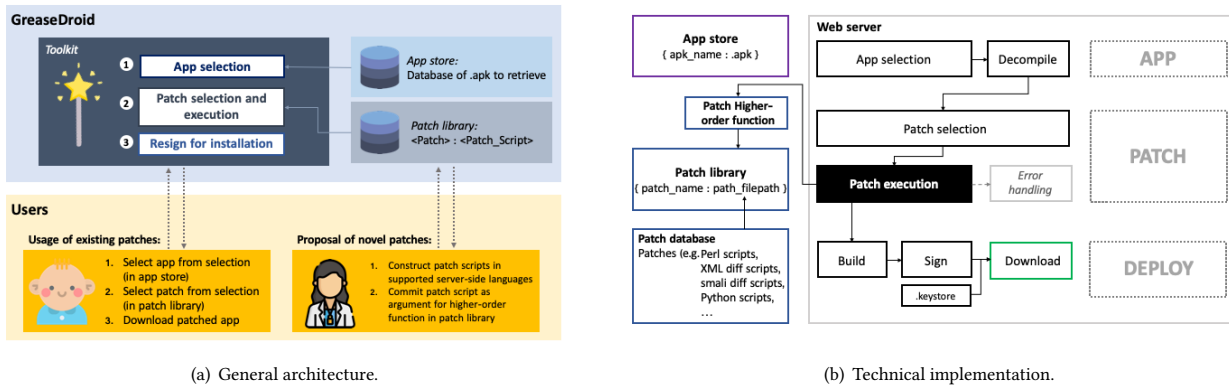


Figure 3: Overview of the GreaseDroid paradigm.

patching based upon such patch scripts. We will explain this method in more detail in the following sections.

Byte masks might be a powerful alternative to patch scripts. These describe patches to apps that do not require decompilation, and instead directly modify the compiled bytecode of apps based on pattern matching. For instance, colours within an app can be changed by simply searching for their hexadecimal representation in the app binary. Similarly, the existing Lucky Patcher tool replaces byte patterns in apps' compiled code to circumvent piracy protections [41]. Since byte masks do not rely on decompilation, they might be easier to deploy to a client-side patching system on the user's phone and might avoid legal issues related to decompilation. We will not study this in more detail in this work because they are more difficult to develop for patch developers.

Ideally, patches crafted by patch developers should be designed to be executable on the patching device (e.g. a Linux-based patching environment may permit for a variety of patching scripts such as Perl or bash). Patches should also take measures to be robust, including compatibility with patches and with updates to the app or operating system.

(3) Re-deployment. After successful app patching, we need to resign in order to install the modified app on the user's device. This re-signing process requires a user-generated certificate to ensure the authenticity of the app on the Android system. It is important that the certificate is unique to the user, to prevent attackers from installing malicious apps on the user's phone. A problem is that signing apps with user-generated certificate makes it impossible to install app updates directly from the Google Play store. GreaseDroid offers an easy solution because the whole patching process can be repeated for any app updates.

4 CLASSES OF PATCHES

GreaseDroid relies on the availability of patches. To populate patches into the patch database, we rely on a network of expert patch developers to craft the patches. Patches can be categorized based on their degree of app-specificity (app-agnostic or app-specific patches) and dark pattern blockage (interface or control flow patches). These classes of patches are explained in the following.

Control Flow patches vs Interface patches. This category of patches is concerned with the types of dark patterns disabled in apps through patching. We refer to Gray *et al.*'s taxonomy of dark patterns [20]: **Interface Interference, Nagging, Forced Action, Obstruction, Sneaking**. Each type of dark pattern can be tackled by either *control flow patches* or *interface patches*. We expect most patches proposed to be interface patches, specifically targeting *Interface Interference* dark patterns. The distribution of dark patterns found in apps is skewed towards *Interface Interference* elements, thus we expect the distribution of patching efforts to tend towards Interface Patches [8, 42].

Interface patches [Interface Interferences] Some of our patches directly change the UI of an app. Specific instances such as *pre-selection* or *hidden information* leverage the attention scope of end-users, so possible patches include the exaggeration of attributes of UI components to draw attention. Another instance, *aesthetic manipulation*, might draw end-user attention to certain components, so possible patches include removing those components or aesthetically-modifying them to be less distracting. We show in Figure 1 the concealment of elements that create distraction for users, such as the *Stories* function and the notification counter. Interface patches are often straightforward develop, requiring the modification of the Android xml layout files only; for example, changing visibility of elements, e.g. changing graphical asset colours to draw attention, rendering transparent or removing UI components. Such modification is similar to modifying html websites, and only requires expertise with app development on Android.

Control flow patches [Nagging, Forced Action, Obstruction, Sneaking] These patches require the modification of program control flow. Patches in this category are more difficult to create. They rely on the modification of high-level assembly code in the *smali* programming language. It is challenging to modify the obfuscated assembly code, as integrity-preserving decompilation into Java for closed-source apps is not guaranteed. Some dark patterns could be disabled by simply removing a function call to a pop-up function (for *nagging*). Some dark patterns are built deep into the function of the app, and may require rewriting how functions execute (e.g. removing token mechanisms for *gamefication*). For example, *forced actions* are in-task interruptions that can only

Patch	Remove location	Patch	Remove stories updates
Attribute	app-agnostic; control flow;	Attribute	app-specific; interface;
Code	<pre>... perl -i -p0e 's/ [^\r\n]+Landroid\/location\/Location; >(getLongitude getLatitude)\\(\\)D[^\r\n]+ move-result-wide v([0-9]+)/' const-wide\\16 v\$2, 0x0"/seg' \$f ...</pre>	<pre>... --<FrameLayout android:background="?coreColorAppBackground" android:layout_width="fill_parent" android:layout_height="wrap_content" +<FrameLayout android:background="?coreColorAppBackground" android:visibility="gone" android:layout_width="fill_parent" android:layout_height="wrap_content" ...</pre>	

Figure 4: Left: An example of an *app-agnostic Control Flow* patch – a Perl script to prevent apps from accessing the longitude and latitude of the current physical user location. Right: An *app-specific Interface* patch – a diff script to remove the *Stories* bar from the Twitter app.

be removed by modifying the control flow logic and refactoring the source code.

App-agnostic vs. app-specific patches. This category of patches is concerned with range of apps that the patch can be applied to. With our provided toolkit, we were able to craft patches for several apps, both (1) *app-agnostic* patches (patches that can be applied to a wide range of applications) and (2) *app-specific* patches (patches that are tailored to a specific application) (Figure 4).

5 IMPLEMENTATION

To test our proposed system of app patching to alleviate dark patterns, we constructed a prototype that enables end-users to apply pre-defined patches to Android apps of their choice. The following discusses this prototype implementation of our *GreaseDroid* paradigm, consisting of end-users and patch developers. Figure 2 shows screenshots of our implementation of the three-step patching process. Figure 3b illustrates our reference implementation of this flow in software.

End-users open the *GreaseDroid* website on their device and select from a pre-selected list of Android apps to patch (Figure 2a). They are shown a list of patches, then choose to apply their selected patches to the app (Figure 2b). When complete, the user is provided a re-signed (Figure 2c) installation file that can be installed on their phone.

Patch developers construct patches in the form of Linux scripts. These *patch scripts* describe how to modify the code and resources of an app. Each patch script comes with additional metadata, including name, description, author, and further robustness parameters (e.g. apps and app versions supported). The use of Linux scripts for patching has multiple advantages. First, it allows individuals with expertise in various programming languages to participate in user-scripting for *GreaseDroid*. Second, Linux-based patches can potentially be run on the Android device itself, reducing the need for an external server and increasing the ease for the end-user. For example, one of our app-agnostic patch scripts applies a regular expression through Perl to remove location access from an app, see Figure 4.

In our *GreaseDroid* implementation, patching is executed on an external server. Patches can be created using common programming languages (e.g. python, Perl, bash) due to server pre-requisites. Patch

developers insert a formatted patching higher-order function into the main patching library that calls and runs the patch script to patch a designated app. We provide interface patches that require modifications to xml layout and smali code files, so we first decode the apk file with apktool, then after calling the patch scripts, we run apktool to build a modified apk from source, and re-sign with user details. We then return a download link for user installation (Figure 2c). To mitigate potential legal issues, we may consider switching from server-side to client-side patching on the user device in future work.

6 CASE STUDY: MODIFICATION OF THE TWITTER APP

To illustrate that *GreaseDroid* can successfully remove dark patterns from apps, we consider the Twitter app on Android. In the main screen of the app, we identified two dark patterns that are used to encourage more user interaction, see Figure 1a. First, the top of the Twitter app shows so-called “Fleets”. These are tweets disappearing after 24 hours, similar to the “Stories” functionality of Facebook, Instagram and Snapchat. Second, the bottom of the app contains a notification counter that informs the users about recent activities of other users. Neither Fleets nor notifications can be fully deactivated within the Twitter app, so some users might want to remove this functionality with *GreaseDroid*, to reduce time spent on Twitter.

We were able to remove the two identified dark patterns from the Twitter app with the help of *GreaseDroid*, by removing the relevant sections from the xml layout files from the decompiled Twitter app (i.e. from those files that describe the UI on Android). It was straightforward to identify the relevant sections of the xml files with one of the run-time app layout inspection tools from the Google Play Store. Additionally, with some more work, we injected code into the smali code of the decompiled Twitter app to refuse attempts to click the notification button, thereby disable access to the notification view of the Twitter app.

The study of the Twitter app demonstrates that it can be straightforward to modify UI components of existing apps. Changing the program flow is more challenging than UI changes because the source code is usually obfuscated, but also possible for patch developers with good programming expertise.

7 DISCUSSION AND FUTURE WORK

Benefits for app developers. An ecosystem of patch developers and patch users is not isolated from the original app developers. Indeed, it may bring benefits for them. The development of patches can serve as a feedback loop, to provide valuable suggestions for the original app developer. With an active network of patch developers and patch adopters, *GreaseDroid* can potentially speed up the app developer's development cycle and reduce costs through crowdsourced software engineering efforts. Indeed, the ability to create patches through *GreaseDroid* might create new financial incentives, wherein app developers reward patch developers, similar to existing bug bounty programs.

An open question is how app developers will perceive increased user patching, whether it will be a moving target leading to an arms race, or whether developers will engage with the community so as to make apps better.

Patch robustness across app updates. App developers regularly release updates to their apps. This poses a particular challenge in designing patches because patches might lead to breakage of apps. However, the amount of change across app updates will usually be rather small. UI resources in Android apps are usually organised around xml layout files, which are easier to modify than compiled source code, as demonstrated by our case study of the Twitter app. In addition, app-agnostic patches should not suffer from issues with app updates. A focus on UI and app-agnostic patches will help patch compatibility with app updates.

Protection against malicious patches. While community-driven user-scripting (1) reduces duplicate patch development and increases patch development efficiency, (2) increases patch quality selection (e.g. higher quality patches given a higher rating by patch adopters), and (3) increases collaborative opportunities within cliques in the patch developer network (e.g. identifying new patches, fixing bugs), a critical challenge is the threat of malicious patches. *GreaseMonkey*'s script market suffered from the existence of a significant number of malicious scripts disguised as benign scripts with abusive vulnerabilities [43]. Though there have been recent developments in automated malware detection [44–50], due to the complexity and heterogeneity of patch scripts, automated approaches might not be sufficiently reliable to mitigate the threat of malicious patches [51–54]. A review system of expert users might be preferable over automated program analysis to reduce malicious scripts. Each patch uploaded to the patch database could be *verified* by a (group of) peer patch developer(s). To ensure authenticity of reviews, patch reviewers could cryptographically sign patches after verification. The reliability of patch review by experts in practice may need to be studied further. Protecting users against malicious code is a great challenge in making *GreaseDroid* available to a wide array of end-users, and will require more consideration and practical experience in future work.

Legality. Using *GreaseDroid* as an end-user might violate the law, particularly the DMCA in the US and the Computer Programs Directive in the EU. Two main issues arise: 1) the distribution of patched apps and 2) the decompilation and disassembly of apps. The decentralised approach of *GreaseDroid* might help to overcome such legal challenges. Whilst developers previously shared patched apps online, *GreaseDroid* separates the distribution of patches from

the patching of apps. Patches are applied at install-time and on the user's device. There is no need to distribute patched apps. Such private modification of apps might be covered by the "fair use" and "right to repair" principles in the US because *GreaseDroid* allows users to remove deficiencies from apps.

Our implementation of patching in *GreaseDroid* does not rely on *decompilation* of the program code, but rather on *disassembly*. This could reduce issues with legislation that bans decompilation. Byte masks – as used in LuckyPatcher – could even remove the need for disassembly, and instead make changes to the app binary directly. Under the EU Computer Programs Directive, decompilation is permitted when "*necessary for the use of the computer program by the lawful acquirer in accordance with its intended purpose, including for error correction*". Enabling disadvantaged users to use essential technologies; promoting research in the public interests; and complying with fundamental human rights laws, including privacy and democratic rights, may well be the intended purpose of many apps.

8 CONCLUSION

This work describes *GreaseDroid*, a framework to reduce dark patterns in mobile apps through app modification, and provide a functional prototype implementation. Using the example of the Twitter app, we illustrate the patch development process and prove the functionality of our method. The successful deployment of community-driven user-scripting to minimize dark patterns in mobile apps can be further pursued with research in digital self-control, app privacy, software law, software engineering practices, and app security and reverse-engineering. Further work should study how our app modification framework can be deployed more widely, particularly mitigating the technical and legal challenges; why users would want to modify their apps; how effective app modification is at helping users gain more agency in their app use. This work aims to open the debate around what choice users should have over their apps, and whether there should be a "right to fair programs", even if this requires changes to existing legislation. Given that users rely on their apps and have currently limited ways to change them, *GreaseDroid* introduces means to change their apps, and negotiate the terms of their apps.

REFERENCES

- [1] Irina Shklovski, Scott D. Mainwaring, Halla Hrund Skúladóttir, and Höskuldur Borgthorsson. 2014. Leakiness and Creepiness in App Space: Perceptions of Privacy and Mobile App Use. In *Proceedings of the 32nd Annual ACM Conference on Human Factors in Computing Systems - CHI '14* (Toronto, Ontario, Canada). ACM Press, New York, NY, United States, 2347–2356. <https://doi.org/10.1145/2556288.2557421>
- [2] Bundeskartellamt. 2019. B6-22/16 (Facebook v Bundeskartellamt).
- [3] Ulrik Lyngs, Kai Lukoff, Petr Slovak, Reuben Binns, Adam Slack, Michael Inzlicht, Max Van Kleeck, and Nigel Shadbolt. 2019. Self-Control in Cyberspace: Applying Dual Systems Theory to a Review of Digital Self-Control Tools. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, United States, 1–18. <https://doi.org/10.1145/3290605.3300361>
- [4] Ulrik Lyngs, Kai Lukoff, Petr Slovak, William Seymour, Helena Webb, Marina Jirotko, Jun Zhao, Max Van Kleeck, and Nigel Shadbolt. 2020. 'I Just Want to Hack Myself to Not Get Distracted': Evaluating Design Interventions for Self-Control on Facebook. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu HI USA, 1–15. <https://doi.org/10.1145/3313831.3376672>
- [5] Reuben Binns, Ulrik Lyngs, Max Van Kleeck, Jun Zhao, Timothy Libert, and Nigel Shadbolt. 2018. Third Party Tracking in the Mobile Ecosystem. In *Proceedings of the 10th ACM Conference on Web Science - WebSci '18* (Amsterdam, Netherlands).

- ACM Press, New York, NY, United States, 23–31. <https://doi.org/10.1145/3201064.3201089>
- [6] Ruha Benjamin. 2019. *Race after Technology: Abolitionist Tools for the New Jim Code*. Polity, Cambridge, United Kingdom.
 - [7] Arunesh Mathur, Gunes Acar, Michael J. Friedman, Elena Lucherini, Jonathan Mayer, Marshini Chetty, and Arvind Narayanan. 2019. Dark Patterns at Scale: Findings from a Crawl of 11K Shopping Websites. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW, Article 81 (Nov. 2019), 32 pages. <https://doi.org/10.1145/3359183>
 - [8] Linda Di Geronimo, Larissa Braz, Enrico Fregnan, Fabio Palomba, and Alberto Bacchelli. 2020. UI Dark Patterns and Where to Find Them: A Study on Mobile Applications and User Perception. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3313831.3376600>
 - [9] Carol Moser, Sarita Y. Schoenebeck, and Paul Resnick. 2019. Impulse Buying: Design Practices and Consumer Needs. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–15. <https://doi.org/10.1145/3290605.3300472>
 - [10] Christine Utz, Martin Degeling, Sascha Fahl, Florian Schaub, and Thorsten Holz. 2019. (Un)Informed Consent: Studying GDPR Consent Notices in the Field. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (London, United Kingdom) (CCS '19). Association for Computing Machinery, New York, NY, USA, 973–990. <https://doi.org/10.1145/3319535.3354212>
 - [11] Dominik Machuletz and Rainer Böhme. 01 Apr. 2020. Multiple Purposes, Multiple Problems: A User Study of Consent Dialogs after GDPR. *Proceedings on Privacy Enhancing Technologies* 2020, 2 (01 Apr. 2020), 481 – 498. <https://doi.org/10.2478/popets.2020-0037>
 - [12] Midas Nouwens, Ilaria Liccardi, Michael Veale, David Karger, and Lalana Kagal. 2020. Dark Patterns after the GDPR: Scraping Consent Pop-Ups and Demonstrating Their Influence. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376321>
 - [13] Than Htut Soe, Oda Elise Nordberg, Frode Guribye, and Marija Slavkovic. 2020. Circumvention by design – dark patterns in cookie consents for online news outlets. arXiv:2006.13985 [cs.HC]
 - [14] Shaomei Wu, Lindsay Reynolds, Xian Li, and Francisco Guzmán. 2019. Design and Evaluation of a Social Media Writing Support Tool for People with Dyslexia. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, Glasgow Scotland UK, 1–14. <https://doi.org/10.1145/3290605.3300746>
 - [15] Sebastian Rieger and Caroline Sindors. 2020. Dark Patterns: Regulating Digital Design. <https://www.stiftung-nv.de/sites/default/files/dark.patterns.english.pdf>
 - [16] Henry H. Wilmer, Lauren E. Sherman, and Jason M. Chein. 2017. Smartphones and Cognition: A Review of Research Exploring the Links between Mobile Technology Habits and Cognitive Functioning. *Frontiers in Psychology* 8 (2017), 605. <https://doi.org/10.3389/fpsyg.2017.00605>
 - [17] Yu-Kang Lee, Chun-Tuan Chang, You Lin, and Zhao-Hong Cheng. 2014. The dark side of smartphone usage: Psychological traits, compulsive behavior and technostress. *Computers in Human Behavior* 31 (2014), 373 – 383. <https://doi.org/10.1016/j.chb.2013.10.047>
 - [18] B.J. Fogg. 2003. *Persuasive Technology: Using Computers to Change What We Think and Do*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
 - [19] Yvonne Rogers, Paul Dourish, Patrick Olivier, Margot Brereton, and Jodi Forlizzi. 2020. The Dark Side of Interaction Design. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '20). Association for Computing Machinery, New York, NY, USA, 1–4. <https://doi.org/10.1145/3334480.3381070>
 - [20] Colin M. Gray, Yubo Kou, Bryan Battles, Joseph Hoggatt, and Austin L. Toombs. 2018. The Dark (Patterns) Side of UX Design. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3173574.3174108>
 - [21] Statista. 2021. Ad blocking user penetration rate in the United States from 2014 to 2021. <https://www.statista.com/statistics/804008/ad-blocking-reach-usage-us/>
 - [22] Matias Martinez, Thomas Durieux, Romain Sommerard, Jifeng Xuan, and Martin Monperrus. 2016. Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset. *Springer Empirical Software Engineering* 22, 4 (2016), 1936–1964. <https://doi.org/10.1007/s10664-016-9470-4>
 - [23] He Ye, Matias Martinez, and Martin Monperrus. 2019. Automated Patch Assessment for Program Repair at Scale. arXiv:1909.13694 [cs.SE]
 - [24] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (Bergamo, Italy) (ESEC/FSE 2015). Association for Computing Machinery, New York, NY, USA, 532–543. <https://doi.org/10.1145/2786805.2786825>
 - [25] Jay Freeman. 2020. Cydia Substrate. <http://www.cydiasubstrate.com/>
 - [26] rovo89. 2020. Xposed Framework. <https://xposed.info/>
 - [27] Yuvraj Agarwal and Malcolm Hall. 2013. ProtectMyPrivacy: Detecting and Mitigating Privacy Leaks on iOS Devices Using Crowdsourcing. In *Proceeding of the 11th Annual International Conference on Mobile Systems, Applications, and Services - MobiSys '13*. ACM Press, Taipei, Taiwan, 97. <https://doi.org/10.1145/2462456.2464460>
 - [28] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation (OSDI'10)*. USENIX Association, Berkeley, CA, United States, 393–407.
 - [29] LuckyPatcher. 2020. Lucky Patcher. <https://www.luckypatchers.com/>
 - [30] Niklas Higi. 2020. apk-mitm. <https://github.com/shroudedcode/apk-mitm/>
 - [31] Jinseong Jeon, Kristopher K. Micinski, Jeffrey A. Vaughan, Ari Fogel, Nikhilesh Reddy, Jeffrey S. Foster, and Todd Millstein. 2012. Dr. Android and Mr. Hide: Fine-Grained Permissions in Android Applications. In *Proceedings of the Second ACM Workshop on Security and Privacy in Smartphones and Mobile Devices - SPSM '12*. ACM Press, Raleigh, North Carolina, USA, 3. <https://doi.org/10.1145/2381934.2381938>
 - [32] Siegfried Rasthofer, Steven Arzt, Enrico Lovat, and Eric Bodden. 2014. DroidForce: Enforcing Complex, Data-Centric, System-Wide Policies in Android. In *2014 Ninth International Conference on Availability, Reliability and Security*. IEEE, Fribourg, Switzerland, 40–49. <https://doi.org/10.1109/ARES.2014.13>
 - [33] Benjamin Davis and Hao Chen. 2013. RetroSkeleton: Retrofitting Android Apps. In *Proceeding of the 11th Annual International Conference on Mobile Systems, Applications, and Services - MobiSys '13*. ACM Press, Taipei, Taiwan, 181. <https://doi.org/10.1145/2462456.2464462>
 - [34] Michael Backes, Sebastian Gerling, Christian Hammer, Matteo Maffei, and Philipp von Styp-Rekowsky. 2014. AppGuard – Fine-Grained Policy Enforcement for Untrusted Android Applications. In *Data Privacy Management and Autonomous Spontaneous Security*, Joaquin Garcia-Alfaro, Georgios Lioudakis, Nora Cuppens-Boulahia, Simon Foley, and William M. Fitzgerald (Eds.). Lecture Notes in Computer Science, Vol. 8247. Springer Berlin Heidelberg, Berlin, Heidelberg, 213–231. https://doi.org/10.1007/978-3-642-54568-9_14
 - [35] Benjamin Davis, Ben S. Armen Khodaverdian, and Hao Chen. 2012. I-ARM-Droid: A rewriting framework for in-app reference monitors for android applications. In *In Proceedings of the Mobile Security Technologies 2012, MOST '12*. IEEE, New York, NY, United States, 1–9.
 - [36] Rubin Xu, Hassen Saidi, and Ross Anderson. 2012. Aurasium: Practical Policy Enforcement for Android Applications. In *21st USENIX Security Symposium (USENIX Security 12)*. USENIX Association, Bellevue, WA, 539–552. https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/xu_rubin
 - [37] AdGuard. 2020. AdGuard. <https://adguard.com/>
 - [38] Konrad Kollnig. 2020. TrackerControl. <https://trackercontrol.org/>
 - [39] evilwombat. 2020. A de-bullshified version of Facebook (less ads, less clutter, less crap). <https://forum.xda-developers.com/t/a-de-bullshified-version-of-facebook-less-ads-less-clutter-less-crap.3586318/>
 - [40] Vanced. 2020. Vanced. <https://vancedapp.com/>
 - [41] Nils Kannengiesser, Johannes Neutze, Uwe Baumgarten, and Sejun and Song. 2016. An Insight to Cracking Solutions and Circumvention of Major Protection Methods for Android. In *International Symposium on Ambient Intelligence and Embedded Systems*. Springer, Heraklion, Crete, Greece, 1–6. http://amies-2016.international-symposium.org/proceedings_2016/Kannengiesser_Neutze_Baumgarten_Song_AmIEs_2016_Paper.pdf
 - [42] Madison Fansher, Shruthi Sai Chivukula, and Colin M. Gray. 2018. #darkpatterns: UX Practitioner Conversations About Ethical Design. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI EA '18). Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3170427.3188553>
 - [43] Steven Van Acker, Nick Nikiforakis, Lieven Desmet, Frank Piessens, and Wouter Joosen. 2014. Monkey-in-the-Browser: Malware and Vulnerabilities in Augmented Browsing Script Markets. In *Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security* (Kyoto, Japan) (ASIA CCS '14). Association for Computing Machinery, New York, NY, USA, 525–530. <https://doi.org/10.1145/2590296.2590311>
 - [44] Lingwei Chen, Shifu Hou, and Yanfang Ye. 2017. SecureDroid: Enhancing Security of Machine Learning-Based Detection against Adversarial Android Malware Attacks. In *Proceedings of the 33rd Annual Computer Security Applications Conference* (Orlando, FL, USA) (ACSAC 2017). Association for Computing Machinery, New York, NY, USA, 362–372. <https://doi.org/10.1145/3134600.3134636>
 - [45] Wei Yang, Deguang Kong, Tao Xie, and Carl A. Gunter. 2017. Malware Detection in Adversarial Settings: Exploiting Feature Evolutions and Confusions in Android Apps. In *Proceedings of the 33rd Annual Computer Security Applications Conference* (Orlando, FL, USA) (ACSAC 2017). Association for Computing Machinery, New York, NY, USA, 288–302. <https://doi.org/10.1145/3134600.3134642>
 - [46] ElMouatez Billah Karbab, Mourad Debbabi, Abdelouahid Derhab, and Djedjiga Mouheb. 2018. MalDozer: Automatic framework for android malware detection using deep learning. *Digital Investigation* 24 (2018), S48 – S59. <https://doi.org/10.1016/j.diin.2018.01.007>

- [47] K. Xu, Yingjiu Li, R. Deng, and K. Chen. 2018. DeepRefiner: Multi-layer Android Malware Detection System Applying Deep Neural Networks. *2018 IEEE European Symposium on Security and Privacy (EuroS&P)* 1, 1 (2018), 473–487. <https://doi.org/10.1109/EuroSP.2018.00040>
- [48] Xin Su, Dafang Zhang, Wenjia Li, and Kai Zhao. 2016. A Deep Learning Approach to Android Malware Feature Learning and Detection. In *2016 IEEE TrustCom/BigDataSE/ISPA*. IEEE, Tianjin, China, 244–251. <https://doi.org/10.1109/TrustCom.2016.0070>
- [49] Hongliang Liang, Yan Song, and Da Xiao. 2017. An end-to-end model for Android malware detection. In *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, Beijing, 140–142. <https://doi.org/10.1109/ISI.2017.8004891>
- [50] Shifu Hou, Aaron Saas, Lifei Chen, and Yanfang Ye. 2016. Deep4MalDroid: A Deep Learning Framework for Android Malware Detection Based on Linux Kernel System Call Graphs. In *2016 IEEE/WIC/ACM International Conference on Web Intelligence Workshops (WIW)*. IEEE, Omaha, NE, USA, 104–111. <https://doi.org/10.1109/WIW.2016.040>
- [51] Kathrin Grosse, Nicolas Papernot, Praveen Manoharan, Michael Backes, and Patrick McDaniel. 2017. Adversarial examples for malware detection. In *Computer Security – ESORICS 2017 - 22nd European Symposium on Research in Computer Security, Proceedings (Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics))*, Simon N. Foley, Dieter Gollmann, and Einar Snekkenes (Eds.). Springer Verlag, Germany, 62–79. https://doi.org/10.1007/978-3-319-66399-9_4
- [52] Sen Chen, Minhui Xue, Lingling Fan, Shuang Hao, Lihua Xu, Haojin Zhu, and Bo Li. 2018. Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach. *Computers & Security* 73 (2018), 326 – 344. <https://doi.org/10.1016/j.cose.2017.11.007>
- [53] Ambra Demontis, Marco Melis, Battista Biggio, Davide Maiorca, Daniel Arp, Konrad Rieck, Igino Corona, Giorgio Giacinto, and Fabio Roli. 2019. Yes, Machine Learning Can Be More Secure! A Case Study on Android Malware Detection. *IEEE Trans. Dependable Secur. Comput.* 16, 4 (July 2019), 711–724. <https://doi.org/10.1109/TDSC.2017.2700270>
- [54] Xiao Chen, Chaoran Li, Derui Wang, Sheng Wen, Jun Zhang, Surya Nepal, Yang Xiang, and Kui Ren. 2020. Android HIV: A Study of Repackaging Malware for Evading Machine-Learning Detection. *IEEE Transactions on Information Forensics and Security* 15 (2020), 987–1001. <https://doi.org/10.1109/TIFS.2019.2932228>